// Sky Vercauteren
// CSU CS 520
// Assignment 6

[[Analysis of Alg.]]

Local Occupancy

Let be the G be 5-cycle graph with vertices {a,b,c,d,e} and edges {ab, bc, cd, de, ea} .

Brute force computations

1. Independant Sets:
    1. {}
    2. {a}
    3. {b}
    4. {c}
    5. {d}
    6. {e}
    7. {a,c}
    8. {a,d}
    9. {b,d}
    10. {b,e}
    11. {e,c}
2. Partition Function:
    1. $Z(\lambda) = \sum_{I.S} \lambda^{length}$
    2. $Z(\lambda) = \lambda_0^0 + \lambda_1^1 + \lambda_2^1 + \cdots + \lambda_7^2 + \ldots \lambda_{11}^2$
    3. $Z(\lambda) = 1 + \lambda + \lambda + \lambda + \lambda + \lambda + \lambda^2 + \lambda^2 + \ldots$
    4. $Z(\lambda) = 1 + 5(\lambda) + 5(\lambda)^2$
3. Expected Size:
    1. $E|I| = \sum_{I}|I|\times Pr(I)$
    2. Where Pr(i) is $\frac{\lambda^{|i|}}{z(\lambda)}$
    3. If lambda was 1 then the partition function puts Z at 11.
        1. $\left(0 * \frac{1}{11}\right) + 5\left(1 * \frac{1}{11}\right) + 5\left(2 * \frac{1}{11}\right)$
        2. $\frac{5}{11} + \frac{10}{11} = \frac{15}{11} = 1\frac{4}{11} =$
    4. Otherwise we would have to write it out:
        1. (using 'z' as a shorthand for Z(\lambda))
        2. $\left(0 * \frac{1}{z}\right) + 5\left(1 * \frac{\lambda}{z}\right) + 5\left(2 * \frac{\lambda^2}{z}\right)$
        3. $\left(5 * \frac{\lambda}{z}\right) + \left(10 * \frac{\lambda^2}{z}\right)$
        4. $\frac{5\lambda}{z} + \frac{10\lambda^2}{z}$

5. And if we sub z for the full thing then
6. $\frac{5\lambda+10\lambda^2}{1+5\lambda+5\lambda^2}$

5. Lol what's the other way of doing this? -- Omg ok the derivative. Got it. Thanks claude.

Local Occupancy

1. The induced subgraphs inside the neighborhood of a (?)
    1. I'm truly not sure I am understanding what this question is asking.
    2. Is this asking for all the subgraphs of nodes *inside* the neighborhood that might be unoccupied if we knew the occupancy of the outside nodes?
        1. So like A has neighbors b, e and not c, d
            1. If C was occupied then B becomes blocked leaving only {e}
            2. If D was occupied then E becomes blocked leaving only {b}
            3. If both are occupied we would get the empty set {} .. But they are connected lol so they cant be. .. So where does the empty set come from?
            4. If neither C or D was occupied then both {e,b} are in the avaliable set
            5. So the set of F are {empty graph, graph of only {e}, graph of only {b}, the graph of {b,e} but b and e aren't connected, so no edges}
2. Why are these the possible F's in all neighborhoods of G?
    1. .. They are not? You lost me. Did I do something wrong above?
    2. Or is this just saying that the graphs are all the same with different labels?
        1. Which makes sense because there are only ever two nodes outside a given neighborhood, so they are either independently occupied or both available. Aha.
3. Z's and E's
    1. Z's
        1. $Z(F_0) = 1$ -- The empty set.
        2. $Z(F_1) = 1 + \lambda$
        3. $Z(F_2) = 1 + \lambda$
        4. $Z(F_3) = 1 + 2\lambda + \lambda^2$ -- both vertices
    2. E's
        1. $E(F_0) = 0$
        2. $E(F_1) = \lambda\frac{1}{1+\lambda}$
        3. $E(F_2) = \lambda\frac{1}{1+\lambda}$
        4. $E(F_1) = \frac{2\lambda+2\lambda^2}{1+2\lambda+\lambda^2} = \frac{2\lambda(1+\lambda)}{(1+\lambda)^2} = \lambda\frac{2}{1+\lambda}$
4. $\lambda > 0 \text{ when } \beta = \frac{1}{p} \text{ and } \gamma = 1$
    1. ??
    2. So lets say lambda is 1
        1. $\frac{1}{Z(F_0)} + E(F_0) = 1 + 0 = 1 == 1$

2. $\frac{1}{Z(F_1)} + E(F_1) = \frac{1}{2} + \frac{1}{2} = 1 == 1$
3. $\frac{1}{Z(F_2)} + E(F_2) = \frac{1}{2} + \frac{1}{2} = 1 == 1$
4. $\frac{1}{Z(F_3)} + E(F_3) = \frac{1}{4} + \frac{2}{2} = \frac{5}{4} > 1$

3. And if lambda was larger than one it follows that the same results would scale accordingly.
4. I feel like I am staring right at it and not seeing it. The following is just me rambling through a process to see if I land anywhere.

```
- We are looking exlusively at a neighborhood because we are plugging in the F terms individ
- The partition function says: Here is a representation of the count of independent sets (in
- 1/Z(F) is what we plug the empty set into the partition function. so 1/Z(F0) is the probab
- So a question that comes up is like "what does the 1 mean"? its not probability it's just
- But then on the other side, if we try to reduce the size of the probability of the empty se

- We choose some coefficient beta to represent the minimum scalar which satisfies the occupan
    - Note that Im still having a hard time conceptualizing what that balance actually is.
- ... If the expected length of the graph of available nodes was 0, then we have to choose so
  - Ok, (again skipping the above) so we want to mysteriously choose some numbers that forc
  - From there I can see the rest of the pieces, The higher the lower bound gets the closer
```

5. -- Are you wanting us to calculate the result of those complex beta and gamma for each F? Or can I just trust you that it satisfies local occupancy because it looks like we make an assumption below?
6. Ok so with $\beta = \frac{4}{1+\log 2}$ and $\gamma = \frac{2\log 2}{1+\log 2}$ then $\beta + \Delta\gamma = \frac{4+\Delta 2\log 2}{1+\log 2}$.
   - Also noting that if $p = \frac{\lambda}{1+\lambda} = \frac{1}{2}$ then $\frac{p}{z} = \frac{1}{2z}$
   1. F_0 is $\left(\frac{4}{1+\log 2} * \frac{1}{2}\right) + \left(\frac{2\log 2}{1+\log 2} * 0\right) = \frac{4}{2+2\log 2} + 0 = \frac{2}{1+\log 2} \approx 1.18$
   2. F_1 is $\left(\frac{4}{1+\log 2} * \frac{1}{4}\right) + \left(\frac{2\log 2}{1+\log 2} * \frac{1}{2}\right) = \left(\frac{1}{1+\log 2}\right) + \left(\frac{2\log 2}{2+2\log 2}\right) = \frac{1+\log 2}{1+\log 2} = 1.$
   3. F_2 is the same $== 1$
   4. F_3 is $\left(\frac{4}{1+\log 2} * \frac{1}{8}\right) + \left(\frac{2\log 2}{1+\log 2} * 1\right) = \left(\frac{1}{2+2\log 2}\right) + \left(\frac{4\log 2}{2+2\log 2}\right) = \frac{1+4\log 2}{2+2\log 2} \approx 1.11$
7. Whereas if $\beta = 2$ and $\gamma = 1$ gives $2 + 1\Delta$:
   1. F_0 is $\left(2 * \frac{1}{2}\right) + 0 = 1$
   2. F_1 is $\left(\frac{1}{2}\right) + \left(\frac{1}{2}\right) = 1$
   3. F_3 is the same $== 1$
   4. And F_4 is $\frac{5}{4}$
   5. Meaning they both hold (ok maybe that was obvious but I had to fight it)
   6. IS it better? $\beta + \Delta\gamma = \frac{4+\Delta 2\log 2}{1+\log 2} > 2 + \Delta$??
      1. At Delta = 2 it simplifies to $4 = 4$ so they are the same.

2. My knowledge of nat logs isn't fresh enough to justify this, but I'm pretty sure you can actually just do the division and drop the log right?
    1. $\frac{4+\Delta 2\log 2}{1+\log 2} = 4 + 2\Delta = 2 + \Delta$ which is literally the same. But I wouldnt be suprised if I missed some algebra there, or if base e changes things.

Optimizing Local Occupancy

1. With $\lambda = \frac{1}{2}$and $\beta = \frac{27}{8}$and $\gamma = \frac{3}{4}$
    1. Noting that $p = \frac{\frac{1}{2}}{1+\frac{1}{2}} = \frac{\frac{1}{2}}{\frac{3}{2}} = \frac{2}{6} = \frac{1}{3}$ and so $\frac{p}{z} = \frac{1}{3z}$
    2. $F_0 = \left(\frac{27}{8} * \frac{1}{3}\right) + \left(\frac{3}{4} * 0\right) = \frac{27}{24}$
    3. $F_1 = \left(\frac{27}{8} * \frac{1}{3\left(\frac{3}{2}\right)}\right) + \left(\frac{3}{4} * \frac{\frac{1}{2}}{\frac{3}{2}}\right) = \left(\frac{27}{8} * \frac{2}{9}\right) + \left(\frac{3}{4} * \frac{1}{3}\right) = \left(\frac{27}{36}\right) + \left(\frac{3}{12}\right) = \frac{3}{4} + \frac{1}{4} = 1$
    4. F_2 is the same $== 1$
    5. F_3 is $\left(\frac{27}{8} * \frac{1}{3\left(\frac{9}{4}\right)}\right) + \left(\frac{3}{4} * \frac{2}{3}\right) = \left(\frac{1}{2}\right) + \left(\frac{1}{2}\right) = 1$
    6. $\beta + 2\gamma = \frac{27}{8} + 2\left(\frac{3}{4}\right) = 27/8 + 12/8 = \frac{39}{8}$
    7. Lower Bound = $\frac{5}{\beta+2\gamma} = \frac{5}{\left(\frac{39}{8}\right)} = \frac{40}{39}$
2. With $\lambda = 1$ and $\beta = \frac{8}{3}$ and $\gamma = \frac{2}{3}$
    1. Noting that $p = \frac{1}{2}$ and so $\frac{p}{z} = \frac{1}{2z}$
    2. F_0 is $\left(\frac{8}{3} * \frac{1}{2}\right) + \left(\frac{2}{3} * 0\right) = \frac{4}{3}$
    3. F_1 is $\left(\frac{8}{3} * \frac{1}{4}\right) + \left(\frac{2}{3} * \frac{1}{2}\right) = \frac{2}{3} + \frac{1}{3} = 1$
    4. F_2 is the same $== 1$
    5. F_3 is $\left(\frac{8}{3} * \frac{1}{8}\right) + \left(\frac{2}{3} * 1\right) = \frac{1}{3} + \frac{2}{3} = 1$
    6. $\beta + 2\gamma = \frac{8}{3} + 2\left(\frac{2}{3}\right) = \frac{12}{3} = 4$
    7. $\frac{5}{\beta+2\gamma} = \frac{5}{4}$
3. With $\lambda = 4$ and $\beta = \frac{5}{4}$ and $\gamma = 1$
    1. Noting that $p = \frac{4}{5}$ and so $p/z = \frac{4}{5(z)}$
    2. F_0 is $\left(\frac{5}{4} * \frac{4}{5}\right) + (1 * 0) = 1 + 0 = 1$
    3. F_1 is $\left(\frac{5}{4}\frac{*4}{5(5)}\right) + \left(1 * \frac{4}{5}\right) = \left(\frac{25}{20}\frac{*4}{25}\right) + \frac{16}{20} = \frac{4}{20} + \frac{16}{20} = 1$
    4. F_2 is the same $== 1$
    5. F_3 is $\left(\frac{5}{4} * \frac{4}{5(25)}\right) + \left(1 * \frac{8}{5}\right) = \left(\frac{125}{100} * \frac{4}{125}\right) + \frac{8}{5} = \frac{1}{25} + \frac{8}{5} = \frac{41}{25}$
    6. $\beta + 2\gamma = \frac{5}{4} + 2(1) = \frac{13}{4}$
    7. Lower bound is $\frac{5}{\left(\frac{13}{4}\right)} = \frac{20}{13}$
4. Compared to the universal parameters when $\beta + 2\gamma = \frac{1+\lambda}{\lambda} + 2$ for some Z_F depending on lambda
    1. So when lambda is 1/2 then beta + delta gamma is $\frac{6}{2} + 2 = 3 + 2 = 5$ and so the lower bound is $\frac{5}{5} = 1$ vs the suggested lower bound of $\frac{40}{39}$. The suggested

bound wins.
2. When lambda is 1 then beta + delta gamma is $2+2=4$ so the lower bound is $\frac{5}{4}$ vs the suggested bound of $\frac{5}{4}$. Tie.
3. When lambda is 4 then beta + delta gamma is $\frac{5}{4}+2=\frac{13}{4}$ making the lower bound $\frac{20}{13}$ against the suggested $\frac{20}{13}$ which is another tie.
4. So I would hypothesize that the optimized lower bounds are only better than the universal ones when lambda is below 1.
    1. .... If lambda represents loosely, the scale of how dense the occupation is, then the only time we have optimized the lower bound is when we let the occupation of the graph be sparse. .. ? Is that a bad way to think about it? I am still trying to conceptualize things.

5. Compare the suggested values with the Expected size at each fugacity and with the greedy bound 5/3
    1. So I'm just gonna make a table.
    2. Also, a point of confusion, The expected sizes I found earlier depended on each subgraph F. Assuming we are looking at the expected size of any independent set from the whole graph based on lambda, how does that relate to the suggested lower bound? What am I missing here.
    3. Also, the greedy lower bound: I'm not sure I understand how we know that is 5/3? Why not just say 2? This is probably fairly trivial. .. And if its better than the local occupancy bound why not just use that? I'm sorry! I actually do remember you talking about this in class, but now that I am understanding how the pieces fir together I have to ask.

```
Ok so from my notes above, the lower bound cant be larger than the expected size. So we are

Still unsure why we are using 5/3 instead of 2 to represent the large independent set?
```

- So the expected size is $\frac{5\lambda+10\lambda^2}{1+5\lambda+5\lambda^2}$ Plugging in for lambda.
    1. $\frac{5\left(\frac{1}{2}\right)+10\left(\frac{1}{2}\right)^2}{1+5\left(\frac{1}{2}\right)+5\left(\frac{1}{2}^2\right)}=\frac{\frac{5}{2}+\frac{10}{4}}{1+\frac{5}{2}+\frac{5}{4}}=\frac{5}{\frac{19}{4}}=\frac{20}{19}$
    2. $\frac{5+10}{1+5+5}=\frac{15}{11}$
    3. $\frac{5(4)+10(4)^2}{1+5(4)+5(4)^2}=\frac{20+10(16)}{1+20+5(16)}=\frac{180}{101}$

Fugacity	Suggested vs	Universal vs	Expected Size vs	greedy
1/2	40/39	5/5	20/19	5/3
1	5/4	5/4	15/11	5/3

Fugacity	Suggested vs	Universal vs	Expected Size vs	greedy
4	20/13	20/13	180/101	5/3

I want to see it better as decimals.

Fugacity	Suggested vs	Universal vs	Expected Size vs	greedy
0.5	1.026	1	1.052	1.67
1	1.25	1.25	1.364	1.67
4	1.53	1.53	1.78	1.67

So at a fixed fugacity, we can only improve the parameters if the fugacity is already low, (like below 1), otherwise, the universal parameters give us the highest lower bound anyway.

At some fixed parameters, we can always increase fugacity to raise the lower bound.

Woah. I just had a cool moment.

But, can we ever increase the lower bound past the greedy bound without burning the cpu trying to do expected size?

Also, wouldn't greedy itself be fairly expensive on large graphs? Like counting all the neighbors of every vertex? I suppose not as much as finding the independent sets, but still not trivial. Hmm. I suppose its at least linear.

Brute Force Dynamics

1. P_a and Q_a
    1. The 11 by 11 matrix Pa describes all the probabilities of going from one state to every other state with respect to updating a single vertex 'a'. Each row and each column represent all the possible independent sets. The element at each cell is the probability of moving from the state given by the row, to the state given by the column. The diagonal then, is the probability of any independent set moving to itself after 'a' is updated. We use 'p' to denote the probability, which we find with $\lambda/(1+\lambda)$. Therefore, all the independent sets related to 'a', will have some relationship to probability 'p' in their cell (namely p or not p given by 1-p) .. (not to be confused with 'p' vs 'np' though... Not altogether unrelated either). Meanwhile all the independent sets who block a will have 1 on their diagonal and all other columns 0, since no update can ever change them in one step. In a nutshell, Pa represents all the possible ways that the graph could permutate with regard to 'a'

over a single update step from wherever it is currently. If we changed lambda to be something besides 1, we would scale all the respective probabilities accordingly.
    1. So if we are at the empty set, and we try to flip a. A must have been unoccupied because we were empty, meaning we can occupy a with p or not with 1-p. If a is already occupied, we can remove it with p and stay where we are with 1-p. If b is occupied, and we try to flip a, it doesnt matter because b blocks it. We have to stay at b. If c is occupied, we either stay at c (1-p) or we add an occupation of a to it (with p) .. Etc.
    2. Damn I really wrote all of that to avoid just filling in the matrix.
2. Similarly Qa is an 11 by 11 matrix that describes probabilities going to and coming from the same independent sets, except instead of listing all the probabilities with regard to the effects of resampling a single vertex 'a', we are listing the probabilities with regard to the effect of resampling the *neighborhood* of 'a'. Essentially instead of a matrix representing all the possible answers to the question "what happens if I flip a coin for this vertex?" we have a matrix with all the possible answers to the question "what happens if I flip coins for the all the nodes connected to 'a', jointly though so they effect each other, according to the hard-core model?".
    1. From the empty set, we resample both {e and b} there is a p in both of those cells and (1-p)^2 in the empty set. (that one depends on both flips). If a is already occupied, it blocks both e and b, so that row is an identity row. From b being occupied then either both flips are heads, both are tails, one is heads or the other is heads. So columns {e} (flip e and unflip b) and {b (itself)} (dont flip e, and dont flip b) will have (1-p)^2 and the joint set {eb} and the empty set {} would both be the product of (p)(1-p). Etc.

2. K^G and K^N
    1. My understanding of Glauber dynamics is sort of just the result of just picking random vertices at every time step. According to the slides the kernel for ordinary random scan Glauber dynamics is $K^G = \frac{1}{n}\sum_u P_u$ where u is some vertex in the graph and n is the number of vertices in the graph. So its the average over all the kernels that each respectively represent the possible (probable) changes to the graph if that vertex was resampled. So as an average, it represents the average probability of any independent set changing to any other independent set if any vertex is chosen. Since 5 out of the 11 independent sets (both rows and columns) in *every* matrix have a 1 on the diagonal and 0's elsewhere, we would expect the distribution to fall towards the diagonal. To clarify, I don't mean that every kernel has the same diagonal, just that every graph has a strong diagonal because 5 of its 11 independent sets cant change. Therefore, the Glauber Kernel will also have a strong diagonal representing the average of the other kernels.
    2. Similar to the Pa and Qa answer, K^N is the same idea, it's the average of its constituent kernels. Also similar to Qa, it includes the consideration of the neighborhood of a vertex. Except this time, instead of focusing on the

neighborhood exclusively, we also take the individual node into account. $K^N = \frac{1}{n}\sum_u Q_u P_u$. This would be akin to taking the average of the result of two sequential resamples (?). E.g. The resulting product of Qu and Pu is the kernel that describes first resampling the neighborhood of u, then resampling u itself. And we are taking the average of those for all u. I'm not entirely sure why we decided to include the individual kernels Pu in this instead of keeping K^N distinct to the neighborhoods like we did with Qu, but it makes sense that now K^N is a useful kernel that represents the average probabilities of any given resample update more generally.

3. I cant talk through these the same way I did Pa and Qa above without actually calculating all the P's and Q's to explain how they would average. So the best I can do is say: "If I had done the work fort all the kernels P and Q then I could give an example of how they would average over the first few rows, which should be trivial. In both cases the largest probabilities are going to be closest to the diagonal."

3. 15 steps by hand.

-- A little javascript to pick a node and flip a coin. -- not sure this will work in a browser, the devcontainer stuff is what I use to build HTML dynamically with js so that it renders in obsidian without having to switch to "read mode". But the idea of the javascript is dirt simple.

```
const box = dv.container.createEl("div");
const v = box.createEl("b");
const c = box.createEl("b");
v.style.cssText = c.style.cssText = "font:bold 2em monospace;margin:0 .5em";
box.createEl("button", {text: "Pick"}).onclick = () => v.textContent = "ABCDE"[Math.floor(Math.ra
box.insertBefore(v, null);
box.createEl("button", {text: "Flip"}).onclick = () => c.textContent = Math.random() < .5 ? "0"
box.appendChild(c);
```

Step	A	B	C	D	E	Pick
0	-	-	-	-	-	{}
1	-	-	-	-	-	C = 0
2	-	X	-	-	-	B = 1
3	-	X	-	-	-	A = 0
4	-	X	-	-	X	E = 1
5	-	X	-	-	X	D = 1

Step	A	B	C	D	E	Pick
6	-	X	-	-	X	A = 1
7	-	X	-	-	X	D = 0
8	-	X	-	-	-	E = 0
9	-	X	-	-	-	E = 0
10	-	X	-	-	-	C = 1
11	-	-	-	-	-	B = 0
12	-	-	X	-	-	C = 1
13	-	-	X	-	-	D = 0
14	-	-	X	-	X	E = 1
15	-	-	X	-	X	B = 1

I have to be honest, this last bullet says "Do this by hand and observe that you don't really need the kernels you just computed!" and I really don't follow.

On one hand, sure, you never actually need the probabilities for anything. Just live your life and shit will happen :D

On the other.. Yeah but those kernels are useful for describing the behavior we just simulated. Further, the simulation holds on the properties claimed by the kernel. E.g. when we try to flip a node that's blocked, we compounded the probability of trying to flip it, with the probability that it was blocked. So the kernel described the behavior appropriately.

I'm so sorry if I missed the point of this last bullet!!